Unreclaimed Memory Characterization for Preempted Reader Threads in Userspace-RCU

Patrick Mountcastle
Electrical and Computer Engineering Department
Virginia Tech
Blacksburg, Virginia
pmountcastle1@vt.edu

Ram Mude
Electrical and Computer Engineering Department
Virginia Tech
Blacksburg, Virginia
ramm@vt.edu

***Abstract*—The Read Copy Update mechanism is a synchronization method that is commonly used today due to its desired lock free read critical section architecture. Generally, an RCU works by using a non-preemption scheme, not interrupting or halting reader threads within their read critical sections. However, in this project we will characterize the behavior of RCU when reader threads are preempted within the read side critical section. Furthermore, we will compare the memory usage between non-preempted and preempted reader threads. For this task we will be utilizing the Userspace RCU, a C programming library, to create a linked-list data structure for testing. We have used a unique solution of combining the call_rcu method within the writer operation and the usleep function, from the standard C library, within the reader operation to simulate preemption. Furthermore, to measure heap memory, we used a Valgrind tool called massif, a heap profiler. From experimentation, we concluded that preemption causes a large increase in memory consumption compared to non-preemption. After 60 seconds, our non-preemption program used roughly 120 kilobytes and our preemption program used over 29 megabytes of data. Based on this characterization of RCU, we claim that when using a preempted RCU synchronization mechanism, the amount of unreclaimed memory grows indefinitely as time increases.**

I. Introduction

A common synchronization method that is widely used today, especially within the Linux kernel, is the Read Copy Update (RCU) mechanism. This synchronization mechanism is popular due to its lockless read critical section which allows for simultaneous execution between all threads. The RCU structure is used mainly when there are many readers and one writer, also known as an updater [1]. This mechanism works by allowing the reader threads to read simultaneously, and when a write operation is invoked, the writer will create a copy of the data object, modify it, and then replace the original with its modified copy atomically [2]. Reader threads in the midst of performing operations would see the original data object, and those who have reached or had a quiescent state will begin to observe the new data object. Once the writer is able to determine that no readers hold reference to the old data object,then it will reclaim that memory [3]. An important condition that allows this memory reclamation scheme to work is that the reader threads are not preempted within their read-critical sections. Non-preemption indicates that reader threads will not be interrupted or paused during the execution of their critical sections when a writer thread is invoked [4]. However, what would happen if reader threads were preempted within their read side critical section? If reader threads were to be preempted, then they would continue to hold the reference on the data structure in use. Thus, we theorize when a writer attempts to reclaim the memory of the old data object it wouldn't be able to because the old data object would be referenced by a preempted reader thread. Therefore, we plan on characterizing the behavior of the Read-Copy-Update synchronization mechanism when the reader threads are preempted inside their read side critical section and determine if unreclaimed memory grows unbound.

II. Related Work

RCU can be implemented in both kernel and user-space, but due to restrictions in the application architecture, research is ongoing for the user level implementation which is not dependent on kernel level environments. In the research titled "Poor Man's RCU" [5], the authors implemented the lock, unlock and synchronize APIs using one mutual exclusion lock and two reader-writer locks which means it is programming language, operating system and CPU architecture independent. This simplicity comes at the cost of efficiency, also the scaling to include more RCU APIs would affect the performance. The efficient and flexible user-level implementation of RCU is proposed in [6]. In this paper, the authors describe RCU implementations based on primitives commonly available to user-level applications. The results of their evaluation shows that Quiescent State Based Reclamation(QSBR) flavor attain highest update rate for read dominated workload. Referring this, we will be using RCU QSBR for the synchronization technique. The preemption of reader threads can not be experimented in the linux kernel environment as there exists precautions in the codebase which prevents the reader threads from preemption. In the paper [7], the authors experimented readers preemption on the virtual machines environment and they observed a significant increase in memory ballooning and performance degradation. Similar to this research paper, this project aims to determine the heap memory consumption when reader preemption occurs. The difference is the implementation of RCU, we are implementing RCU in user-space and evaluating the heap memory in the user-level.

III. Proposed Approach

The characterization of behavior of RCU when the reader threads are preempted in the read side critical section is evaluated on the userspace implementation of RCU, that is Userspace-RCU. This library provides functionality to implement a Read Copy Update synchronization mechanism through its limited, but powerful, API method calls. We will be using the C programming language to implement the userspace rcu implementation since it was developed for this language. Additionally, we will be using the Linux architecture along with the GCC compiler to build and execute our programs. This system design was chosen because the Userspace RCU library has been extensively tested on the Linux architecture and, therefore, is best suited to work with our chosen software library [8]. Additionally, inside the Userspace RCU library there are multiple flavors of RCU to choose from. We decided to use the quiescent state based reclamation because it allows us to manually call for a quiescent state on the reader version, which indicates some memory is no longer being referenced and can be reclaimed. From the Userspace RCU API, here are some of the core method calls that we will be using:

1) void rcu_read_lock(void).
2) void rcu_read_unlock(void)
3) T rcu_dereference(T)
4) void synchronize_rcu(void)
5) call_rcu(struct rcu_head *head, void (*func)(struct rcu_head *head))
6) rcu_assign_pointer(a, b)

These method calls are going to be used to implement a userspace rcu linked-list. This data structure will be used to study how memory reclamation behaves when using preemption and non-preemption. The first two methods, lock and unlock (1)(2), will be used in the read side to indicate when read critical sections begin and end. The de-reference method (3) will be used to get data from linked-list pointers that have been assigned within the userspace rcu structure using the assign pointer method call (6). The synchronize method (4) will be used to ensure non-preemption in our linked-list. This method call is a blocking function that ensures all read threads have finished their critical sections before the writer thread is able to call free() to reclaim memory. So, this ensures that the reader threads are not preempted within their critical section. [9] On the other hand, the call rcu method (5) will be used to simulate preemption within our program because it is a non-blocking call. Non-blocking indicates that the method call will not wait for all reader threads to finish there read critical section to execute. [9] However, if a reference to the data structure is still in use, then the call function will wait to free that memory until no reference is being held anymore. In order to simulate preemption within our program we will be using the call_rcu function with the usleep function from the standard C library. The usleep method call sleeps a thread for a duration of time set by the user. Our approach will be to implement call_rcu within our linked-list add node, the writer function, and usleep inside the linked-list contains, our reader function. What will happen is that once a writer thread creates a new node, it would immediately call call_rcu to free that node. But, we are going to use usleep to sleep the reader threads inside the read critical section for over a minute. Our theory is that even though call_rcu is being invoke, no memory reclamation will occur since those reader threads still hold reference to the old data while asleep. So, if the memory is unreclaimed then the total memory being used in the heap will grow significantly as time increases. Altogether, we will be utilizing the Userpsace RCU library to compare how memory usage grows with non-preemption and preemption within a linked-list and determine if unclaimed memory grows indefinitely when using preemption.

IV. Evaluation

The metric for the evaluation is the amount of unreclaimed memory in heap. For this purpose, a heap profiler Massif tool [10] from Valgrind is utilized. Massif measures how much heap memory the given program uses. It offers detailed snapshots of memory consumption over time and tracks memory allocations, including those executed with malloc or new. All of Massif's profiling data is written to a file named massif.out.<pid>. Here pid is the process ID of the running program. The information gathered by massif can be viewed in an easy-to-read form using "ms_print" which is also a Valgrind's tool. A graph is created by ms_print which shows the consumption of memory over the program's execution.

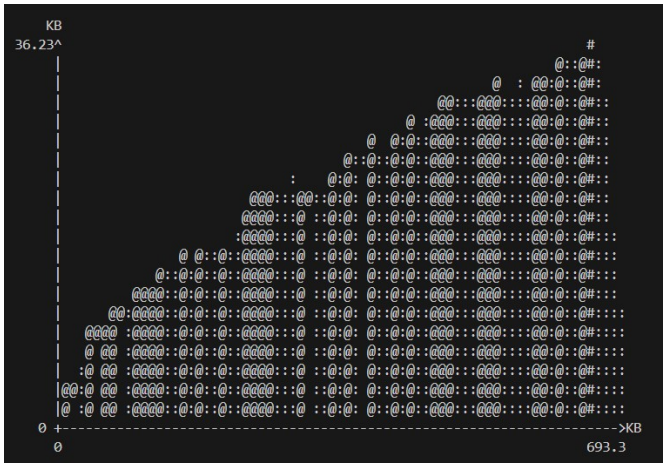


Fig. 1. Memory consumption graph for Linked-list with non-preempted reader threads with 10 total threads running for 10 seconds

Figure 1 shows the heap memory consumption graph created by ms_print with the information gathered by Valgrind's Massif tool, in which x axis is time and y axis is the heap memory consumed by the program. In this graph, each vertical bar represents a snapshot. A snapshot is the measurement of memory usage at a certain point in time. Bars made up of ':' characters represent normal snapshots which indicate that only basic information is recorded for them. Bars made up of '@' characters represent detailed snapshots which indicate that the location of allocations are recorded for them. In a graph, there

is atmost one bar made up of '#' characters which represents the peak detailed snapshot. This peak snapshot records the point in time where the heap memory consumption was the greatest.

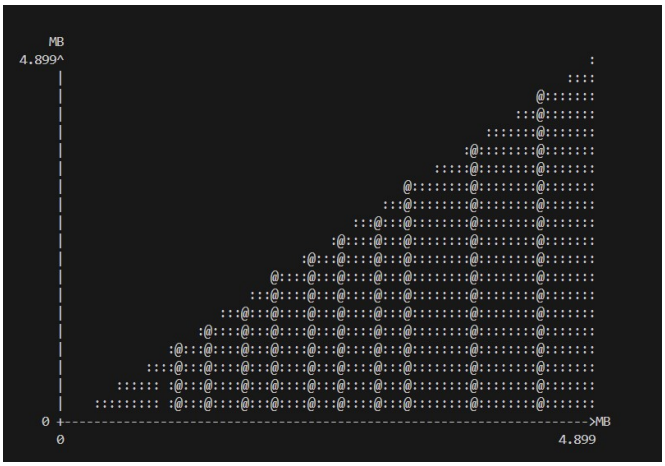


Fig. 2. Memory consumption graph for Linked-list with preempted reader threads with 10 total threads running for 10 seconds

The program is considered to be linkedlist data structure with 10 threads which performs 80% reads and 20% writes. This means that eight threads would call contains function to check for a random number is present in the linkedlist or not. The remaining two threads would call add or remove function to insert a random number in the linkedlist or delete a random number if present in the linkedlist. When the reader threads are not preempted, that is considered as a baseline and then program is modified to preempt the reader threads in the read side critical section. Now there are two programs, baseline and modified(or preempted) program. Both programs are coded so that the threads are killed after 60 seconds, the reason behind this is that, as the reader threads are preempted in the modified program, it requires more than this time to complete and exit. On the other hand, baseline program completes within this time. When figure 1 and figure 2 are compared, the modified program in not run till completion, hence massif cannot determine the peak heap memory consumed, so bar containing '#' characters is not present unlike the baseline program.

Figure 3 delineates the increasing consumption of heap memory for baseline(non-preempted reader threads) and modified(preempted reader threads) program as a function of time. The amount of heap memory consumed by modified program is in the order of Megabytes which is large as compared to the baseline, the difference is so large that the increase in heap memory consumption of baseline program–which is in the order of Kilobytes– over time is not visible. If the scale of heap memory consumption is changed from normal to logarithmic with the base 2, consumption of both programs can be portrayed in figure 4.

Figure 4 shows that for the baseline program, the heap memory consumption ranges from 33 KB for 5 seconds to

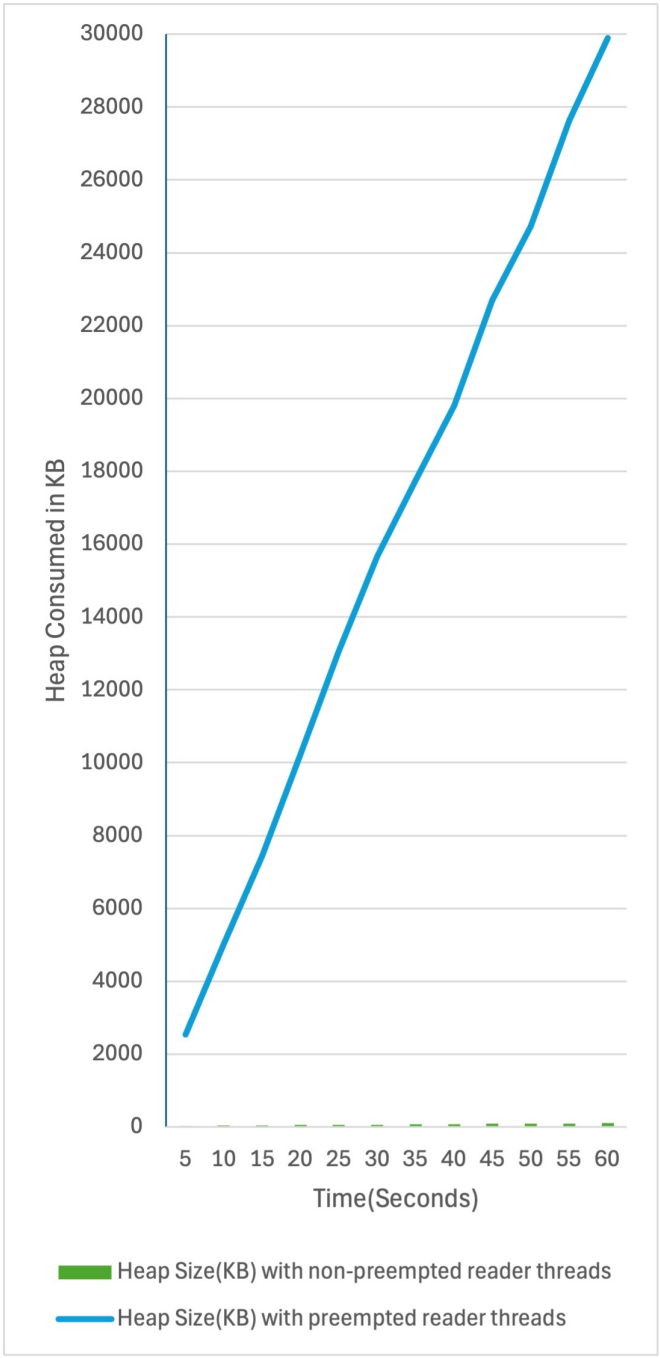


Fig. 3. Normal scale: Heap memory consumption for 80% read and 20% write

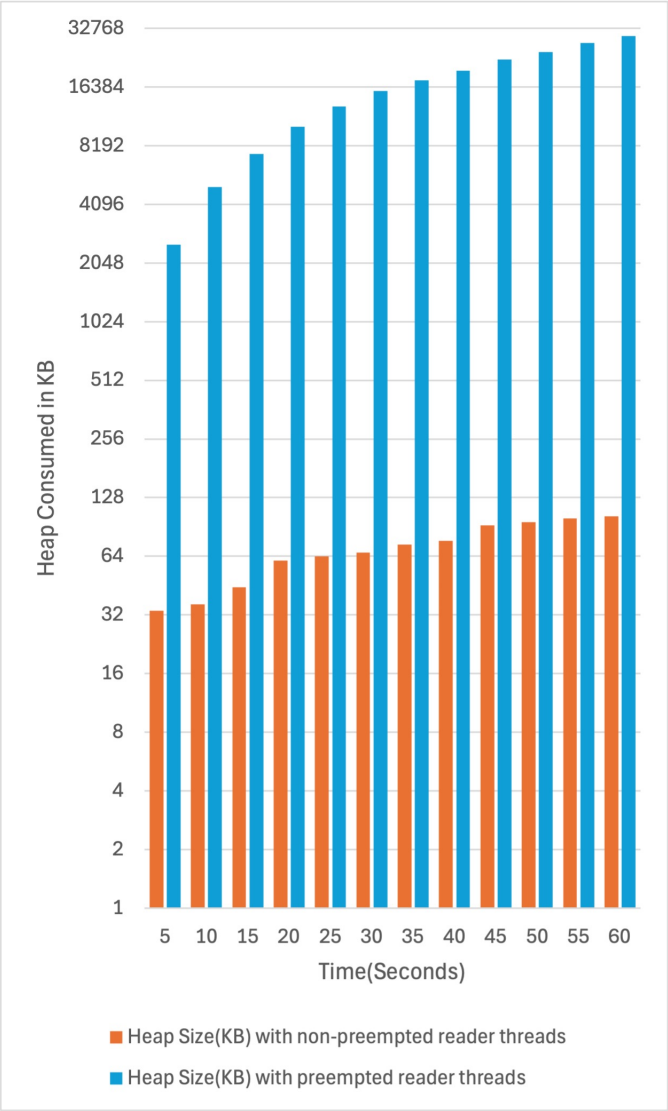


Fig. 4. Logarithmic scale: Heap memory consumption for 80% read and 20% write

120 KB for 60 seconds, which indicates that it does not increase significantly for the given program. This shows that the writer threads are reclaiming the memory of nodes which are to be deleted. As the reader threads are not preempted, that is, not holding references to the nodes to be deleted, writer threads call free function to reclaim the memory allocated to those nodes. For the modified program, the heap memory consumption ranges from 2 MB to 30 MB, which indicates that it is increasing almost linearly, which is clearly visible in figure 3. Consequently, if this modified program would run for above 1 minute, heap consumption keeps increasing. This shows that the writer threads are not able reclaim the memory of the nodes which are to be deleted. The reason being the preemption of reader threads in the read side critical section. As the reader threads sleep after calling rcu_read_lock(), and do not call rcu_read_unlock(), the writer threads can not reclaim the memory of that node because reader threads are still holding references to those nodes. As well as, due to the behavior of writer threads, they should wait for a quiescent state after it confirms that none of the reader threads hold reference to the node to be deleted. For all the reader threads to release the references to the node, reader threads have to call rcu_read_unlock(), which they do after waking up from sleep. Eventually the rate of reclamation of memory of the nodes to be deleted becomes very less compared to the rate of memory allocation for the addition of new nodes, this leads to almost linear heap memory consumption for the modified program.

V. Conclusion

In this project, the behavior of Read-Copy Update(RCU) is characterized when the readers are preempted in the read side critical section. For the implementation of RCU, Userspace-RCU is utilized which is a library in C language. The preemption of reader threads is executed by calling usleep() function. This introduces delay in the call for read side unlock critical section which means that the reader threads hold references to the nodes to be deleted for comparable large amount of time. This leads to the writer threads unable to reclaim the heap memory of these nodes. As a result, the rate to heap memory consumption– new memory allocation for new nodes– is much larger than the rate of heap memory reclamation for the nodes to be deleted. The unreclaimed heap memory increases linearly with time as the program continues to execute. Overall, we can characterize the behavior of a preempted RCU's memory to grow indefinitely as time increases. Looking ahead, there are a couple ways we would want to build off our findings in this project. One, we would compare the results from a preempted RCU with the use of hazard pointers. Hazard pointers are pointers that communicate to a writer when a reader is accessing a reference to a data object. Therefore, we would characterize the behavior of a preempted RCU with a preempted RCU using Hazard pointers to determine if hazard pointers have a positive or negative affect within the preemption problem. Furthermore, we would continue our work by attempting to come up with a solution for the preemption problem. We would take inspiration from epoch based memory reclamation and determine if there is a way to incorporate its memory reclamation scheme within a preempted RCU.

References

[1] GeeksforGeeks, "Read-Copy Update (RCU)," GeeksforGeeks, Jun. 13, 2019. https://www.geeksforgeeks.org/read-copy-update-rcu/
[2] "Basic concept of RCU: Read, Copy, Update - free5GC," Free5gc.org, 2023. https://free5gc.org/blog/20231129/20231129/
[3] "What is RCU, Fundamentally? [LWN.net]," lwn.net. https://lwn.net/Articles/262464/
[4] GeeksForGeeks. "Preemptive and Non-Preemptive Scheduling - GeeksforGeeks." GeeksforGeeks, 13 Nov. 2018, www.geeksforgeeks.org/preemptive-and-non-preemptive-scheduling/.
[5] Ramalhete, Pedro, and Andreia Correia. "POSTER: Poor Man's URCU." ACM SIGPLAN Notices 52, no. 8 (2017): 451-452.

[6] M. Desnoyers, P. E. McKenney, A. S. Stern, M. R. Dagenais and J. Walpole, "User-Level Implementations of Read-Copy Update," in IEEE Transactions on Parallel and Distributed Systems, vol. 23, no. 2, pp. 375-382, Feb. 2012, doi: 10.1109/TPDS.2011.159.
[7] Prasad, Aravinda, K. Gopinath, and Paul E. McKenney. "The RCU-Reader Preemption Problem in VMs." In 2017 USENIX Annual Technical Conference (USENIX ATC 17), pp. 265-270. 2017.
[8] "Userspace RCU." Liburcu.org, 2024, liburcu.org/.
[9] "The User-Space RCU API [LWN.net]." Lwn.net, 2024, lwn.net/Articles/573439/.
[10] massif: A heap profiler. Valgrind. (n.d.). https://valgrind.org/docs/manual/ms-manual.html